Interactive Quiz Application

Project Report
Built with React, Spring Boot and MySQL

Developed by: Ch. Lipika

October 2026

Project 1 - Full Stack (React, Spring Boot, MySQL)

Contents

1. About This Project 3

2. How the Application Works 4

3. How I Built It, Step by Step 5

4. The Server Addresses (API) 8

5. How to Run the Project (Windows) 8

6. Problems I Faced and How I Solved Them 9

7. Saving the Code on GitHub 9

8. Putting the App Online (Deployment) 9

9. Conclusion 10

Appendix A. Complete Source Code 11

Appendix B. README.md (full text) 30

1. About This Project

This project is a **quiz website**. A person can create an account, log in, answer multiple-choice questions, see their score at the end, and look back at which answers were right or wrong. A special user called the **admin** can add, change and delete the quiz questions.

What the website can do

- **For a quiz taker:** register and log in, choose how many questions to answer, move to the next or previous question, submit, see the score, review every answer with an explanation, and view past scores.
- **For the admin:** open a special Admin page to add new questions, edit them, or delete them.
- **Safety:** passwords are stored in scrambled form, only logged-in people can use the quiz, and only the admin can change questions.

Technologies used (in simple words)

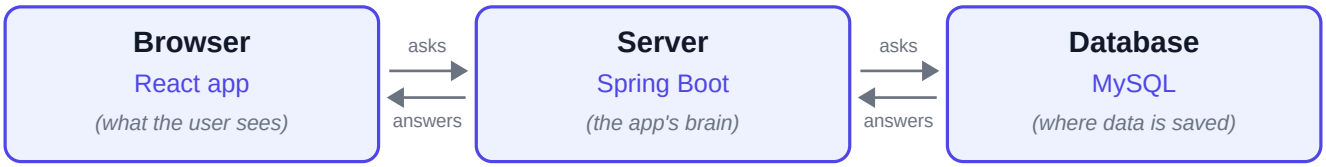
Part	Technology	What it does
Frontend	React	Builds the pages and buttons that the user sees and clicks
Backend	Spring Boot (Java)	Runs behind the scenes: checks logins, picks questions, calculates scores
Database	MySQL	Saves users, questions and quiz results permanently
Tools	Maven, Node.js, Git	Maven builds the Java code, Node.js runs React, Git saves versions of the code

Words you will see in this report

Term	Meaning
API / REST API	A set of addresses (URLs) the frontend calls to ask the backend for something, for example "give me 5 questions"
JWT (token)	A small digital pass the server gives you when you log in. You show it with every later request so the server knows who you are
BCrypt	A method that scrambles a password so the real password is never saved
Role	The type of user: USER (normal) or ADMIN (can manage questions)
CORS	A browser safety rule about one website calling another address. It caused an error during setup (see section 6)
Proxy	A helper that passes the browser's requests on to the backend, which avoids the CORS problem while developing

2. How the Application Works

The application has three parts that talk to each other:



The **browser** shows the pages. When it needs data (for example the questions), it **asks the server**. The server gets the data from the **database** and sends the answer back.

What happens when someone takes a quiz

#	What the user does	What happens behind the scenes
1	Logs in with username and password	The server checks the password and returns a token (the digital pass)
2	Clicks Start Quiz	The server picks random questions from the database and sends them **without** the correct answers
3	Selects an answer for each question	The browser remembers the choices
4	Clicks Submit	The browser sends all choices to the server
5	-	The server compares the choices with the correct answers, counts the score and saves the result
6	Sees the score and review	The server sends back the score and the right/wrong list with explanations

Why the score is calculated on the server: if the browser knew the correct answers, a user could look them up. Keeping the answers on the server makes the quiz fair.

Pages of the website

Page	Who can open it	What it shows
Login / Register	Everyone	One form that switches between logging in and creating an account
Home	Logged-in users	Choose 3, 5, 10 or 15 questions and press Start Quiz
Quiz	Logged-in users	One question at a time with a progress bar and Previous / Next buttons
Result	Logged-in users	Your score, then every question with the correct answer and an explanation
My Scores	Logged-in users	A table of all your earlier attempts
Admin	Admin only	A form to add or edit a question, and a table of all questions with Edit / Delete

3. How I Built It, Step by Step

The project brief has eight steps. Below, each step is explained simply: what the step means, what I did, and where to find the result.

Step 1 - Project Planning

What it means: deciding what the app will do and how it will look before writing any code.

- I listed the features for the two kinds of users: quiz takers and the admin (see section 1).
- I planned the pages: Login/Register, Home, Quiz, Result, My Scores and Admin (see section 2).
- I decided the admin page must be hidden from normal users.

Step 2 - Setting up the Development Environment

What it means: installing the programs needed to build and run the project on my Windows computer.

Program	Version	Why it is needed
Java JDK (Temurin)	17.0.20.1	Runs the Spring Boot backend
Apache Maven	3.10.0	Downloads libraries and builds/runs the Java project
Node.js and npm	24.21.0 / 11.19.0	Runs the React frontend
MySQL Server	8.0.43	The database
Git	2.56.0	Saves the code history and uploads it to GitHub

- I checked each program by running a version command, for example `mvn -v` and `node -v`.
- My PC also had an old Java 8. Spring Boot 3 needs Java 17, so I installed JDK 17 and set `JAVA_HOME` to it.
- I added Maven and MySQL to the Windows PATH so their commands work in PowerShell.
- The backend is a Maven project (`backend/pom.xml`) and the frontend is a Vite + React project (`frontend/package.json`).

Step 3 - Database Design

What it means: deciding which tables store the information, like folders in a filing cabinet.

Table	What it stores	Main columns
users	People who can log in	username, password (scrambled), role
questions	The quiz questions	text, correct answer number, category, explanation
question_options	The answer choices of each question	which question, position, option text
quiz_results	Every quiz attempt	which user, score, total, date and time

- One user can have many results, and one question has many options.
- I did not write the tables by hand: Spring Boot creates them automatically the first time it starts (`ddl-auto=update`). The file `database/schema.sql` shows the same design for reference.

Step 4 - Backend Development (Spring Boot)

What it means: writing the server that handles logins, questions and scores. The code is split into small groups of files, each with one job:

Folder	Job	Files
model	Describes the data (one file per table)	User, Question, QuizResult

Folder	Job	Files
repo	Reads and saves data in MySQL (no SQL needed)	UserRepository, QuestionRepository, QuizResultRepository
dto	Defines the shape of data sent to/from the browser	Dtos
service	The main logic: pick random questions, calculate the score	QuizService
web	The addresses (endpoints) the browser calls	AuthController, QuizController, AdminQuestionController
security	Creates and checks the login tokens	JwtService, JwtAuthFilter
config	Security rules, and first-time data	SecurityConfig, DataInitializer

- The **controllers** receive requests, the **service** does the work, and the **repositories** talk to the database.
- When the backend starts for the first time it creates the admin account and six sample questions (`DataInitializer`).
- All the endpoints are listed in section 4.

Step 5 - Frontend Development (React)

What it means: building the pages the user sees and connecting them to the backend.

File	What it does
`App.jsx`	Decides which page to show for each web address, and blocks pages the user is not allowed to see
`AuthContext.jsx`	Remembers who is logged in (name, role and token)
`api.js`	Sends requests to the backend and automatically adds the login token
`Login.jsx`, `Home.jsx`	Login/Register form and the start screen
`Quiz.jsx`	Shows questions one by one and sends the answers when you submit
`Result.jsx`, `History.jsx`	Score with answer review, and the list of past scores
`Admin.jsx`	Add, edit and delete questions
`Navbar.jsx`	Top bar with links, username and Logout (Admin link only for admins)
`styles.css`	Colors, spacing and layout

Step 6 - Security and User Authentication

What it means: making sure only the right people can do the right things.

- **Passwords** are scrambled with BCrypt before saving, so nobody can read them from the database.
- **Login token:** after a correct login the server gives a JWT token (valid for 24 hours). The browser sends it with every request.
- **Access rules** (`SecurityConfig`): anyone can log in or register; every other page needs a login; the admin endpoints need the ADMIN role.
- **No self-made admins:** people who register always become normal users. The only admin is created at startup (username `admin`).
- **Hidden answers:** the quiz endpoint never sends the correct answers before you submit.
- **Checked input:** the server rejects empty or invalid data. Secrets such as the database password come from environment variables, not from the code.

Step 7 - Testing and Debugging

What it means: checking that everything works and fixing problems.

Automated test: `QuizServiceTest` checks that correct answers are counted and skipped questions are marked wrong. It runs with `mvn test`.

Manual checklist (run these after starting the app; the first item was confirmed during development):

- The backend starts and shows `Started QuizApplication`, and the login API returns a token for the admin.
- Admin can add, edit and delete a question.
- A new user can register, take a quiz, see the score and review, and see the score history.
- A normal user cannot open the Admin page.

The problems I met and how I fixed them are explained in section 6.

Step 8 - Documentation and Version Control

- I wrote a **README.md** that explains the features, setup, API endpoints and deployment (Appendix B).
- I use **Git** to save versions of the code and upload it to **GitHub** (section 7).

4. The Server Addresses (API)

These are the addresses the React app calls. "Access" says who may use each one. After logging in, the browser sends the token in the header `Authorization: Bearer <token>`.

Method	Address	Access	What it does
POST	/api/auth/register	Everyone	Create an account
POST	/api/auth/login	Everyone	Log in and receive a token
GET	/api/quiz/questions?limit=5	Logged in	Get random questions (without answers)
POST	/api/quiz/submit	Logged in	Send answers, receive score and review
GET	/api/quiz/results	Logged in	Get your past scores
GET	/api/admin/questions	Admin	List all questions with answers
POST	/api/admin/questions	Admin	Add a question
PUT	/api/admin/questions/{id}	Admin	Change a question
DELETE	/api/admin/questions/{id}	Admin	Delete a question

Example of the information used to add a question:

```
{ "text": "Which hook manages state in React?",
  "options": ["useEffect", "useRef", "useState", "useMemo"],
  "correctIndex": 2, "category": "React", "explanation": "useState returns a value and a setter." }
```

`correctIndex` counts from 0, so 2 means the third option ("useState").

5. How to Run the Project (Windows)

You need two PowerShell windows: one for the backend and one for the frontend. Make sure MySQL is running first.

Window 1 - Backend

```
cd quiz-app\backend
$env:DB_USER='root'
$env:DB_PASSWORD='your_mysql_password'
mvn spring-boot:run
```

Wait until you see `Started QuizApplication`. Keep this window open. The database tables and the admin account are created automatically.

Window 2 - Frontend

```
cd quiz-app\frontend
copy .env.example .env
npm install
npm run dev
```

Open **http://localhost:5173** in your browser. Log in as `admin` with password `admin123`, or create a new account.

Run the automated test

```
cd quiz-app\backend
mvn test
```

6. Problems I Faced and How I Solved Them

Setting up a full-stack project always brings some errors. These are the ones I met and the fix for each.

Problem	Why it happened	How I fixed it
`java -version` showed version 1.8	An old Java 8 was found first on my PC	Installed JDK 17 and pointed `JAVA_HOME` to it (Maven uses `JAVA_HOME`)
`mvn` command not found	Maven was unzipped into a folder inside a folder	Added the correct `...\apache-maven-3.10.0\bin` folder to PATH
`npm` blocked in PowerShell	Windows blocks scripts by default	Ran `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
`mysql` command not found	MySQL's folder was not in PATH	Added `C:\Program Files\MySQL\MySQL Server 8.0\bin` to PATH
"No plugin found for prefix 'spring-boot'"	I ran Maven outside the `backend` folder, so it found no `pom.xml`	Opened PowerShell inside the `backend` folder first
"Port 8080 already in use"	Another program on my PC used port 8080	Moved the backend to port 8081 (`server.port=${SERVER_PORT:8081}`)
"Port 8081 already in use"	The old backend was still running	Found it with `netstat -ano | findstr :8081` and stopped it with `taskkill /PID <number> /F`
Login showed "Request failed" (browser error 403 on OPTIONS)	The browser's CORS safety check was rejected by the backend	Added a Vite proxy so the browser only talks to port 5173, set `VITE_API_URL=/api`, and allowed localhost in the backend's CORS settings

Tip: after changing `.env` or `vite.config.js`, stop the frontend with Ctrl+C and run `npm run dev` again. Vite only reads these files when it starts.

7. Saving the Code on GitHub

- Create an empty repository on github.com and write your name in the repository description.
- Open PowerShell in the project folder and run the commands below, using your own repository address.
- Tip: make a few separate commits (backend, frontend, README) so your history shows the work step by step.

```
git init
git add .
git commit -m "Interactive quiz application"
git branch -M main
git remote add origin https://github.com/<username>/<repo>.git
git push -u origin main
```

8. Putting the App Online (Deployment)

Running on your own computer is called "development". To let other people use the app, host it online:

Backend

- Build it: `cd backend` then `mvn clean package`.
- Run it: `java -jar target/quiz-backend-1.0.0.jar`.
- Set these settings as environment variables: `DB_USER`, `DB_PASSWORD`, `JWT_SECRET` (a long random text, 32+ characters), `ADMIN_PASSWORD`, `CORS_ORIGIN` (your website address) and `SERVER_PORT`.
- Use a hosted MySQL database and point `spring.datasource.url` to it.

Frontend

- Set `VITE_API_URL` to the online address of the backend, for example `https://api.example.com/api`.
- Build with `npm run build`, then upload the `dist` folder to a host such as Netlify, Vercel or Nginx.
- The Vite proxy only works on your computer. Online, the backend's `CORS_ORIGIN` must match the website address.

9. Conclusion

This project shows a complete full-stack application: a React frontend, a Spring Boot REST API with secure login, and a MySQL database. Users can take quizzes and review their results, and the admin can manage the questions.

Ideas for the future

- A timer for each question
- Filter questions by category
- A leaderboard of top scores
- Pages (pagination) in the admin question list
- Docker files to run everything with one command

Appendix A. Complete Source Code

Every source file of the project is listed below with its path (relative to the `quiz-app` folder). Emoji characters in the user interface are shown as `\u{CODEPOINT}` because PDF fonts cannot print them.

Backend - build and configuration

backend/pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.
      xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.5</version>
    <relativePath/>
  </parent>
  <groupId>com.quizapp</groupId>
  <artifactId>quiz-backend</artifactId>
  <version>1.0.0</version>
  <properties><java.version>17</java.version></properties>
  <dependencies>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-web</artifact
      Id></dependency>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-data-jpa</art
      ifactId></dependency>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-security</art
      ifactId></dependency>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-validation</a
      rtifactId></dependency>
    <dependency><groupId>com.mysql</groupId><artifactId>mysql-connector-j</artifactId><scope>runtime</sc
      ope></dependency>
    <dependency><groupId>org.projectlombok</groupId><artifactId>lombok</artifactId><optional>true</optio
      nal></dependency>
    <dependency><groupId>io.jsonwebtoken</groupId><artifactId>jjwt-api</artifactId><version>0.12.5</vers
      ion></dependency>
    <dependency><groupId>io.jsonwebtoken</groupId><artifactId>jjwt-impl</artifactId><version>0.12.5</ver
      sion><scope>runtime</scope></dependency>
    <dependency><groupId>io.jsonwebtoken</groupId><artifactId>jjwt-jackson</artifactId><version>0.12.5</
      version><scope>runtime</scope></dependency>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-test</artifac
      tId><scope>test</scope></dependency>
  </dependencies>
  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
        <configuration><excludes><exclude><groupId>org.projectlombok</groupId><artifactId>lombok</artifa
      ctId></exclude></excludes></configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

backend/src/main/resources/application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/quizdb?createDatabaseIfNotExist=true&useSSL=false&serv
      erTimezone=UTC
spring.datasource.username=${DB_USER:root}
spring.datasource.password=${DB_PASSWORD:root}
spring.jpa.hibernate.ddl-auto=update
spring.jpa.open-in-view=false
server.port=${SERVER_PORT:8081}

# JWT (secret must be >= 32 characters; override with env var in production)
app.jwt.secret=${JWT_SECRET:change-this-secret-to-a-long-random-string-123456}
app.jwt.expiration-ms=86400000

# CORS
app.cors.allowed-origin=${CORS_ORIGIN:http://localhost:5173}

# Seeded admin account (created on first run)
```

```
app.admin.username=${ADMIN_USER:admin}
app.admin.password=${ADMIN_PASSWORD:admin123}
```

Backend - application, models, repositories

backend/src/main/java/com/quizapp/QuizApplication.java

```
package com.quizapp;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class QuizApplication {
    public static void main(String[] args) {
        SpringApplication.run(QuizApplication.class, args);
    }
}
```

backend/src/main/java/com/quizapp/model/User.java

```
package com.quizapp.model;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Table(name = "users")
@Getter @Setter @NoArgsConstructor
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 50)
    private String username;

    @Column(nullable = false)
    private String password; // BCrypt hash

    @Column(nullable = false, length = 20)
    private String role; // USER or ADMIN
}
```

backend/src/main/java/com/quizapp/model/Question.java

```
package com.quizapp.model;

import jakarta.persistence.*;
import lombok.*;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "questions")
@Getter @Setter @NoArgsConstructor
public class Question {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 500)
    private String text;

    @ElementCollection(fetch = FetchType.EAGER)
    @CollectionTable(name = "question_options", joinColumns = @JoinColumn(name = "question_id"))
    @OrderColumn(name = "position")
    @Column(name = "option_text", nullable = false, length = 300)
    private List<String> options = new ArrayList<>();

    @Column(nullable = false)
    private int correctIndex;

    private String category;

    @Column(length = 500)
    private String explanation;
}
```

backend/src/main/java/com/quizapp/model/QuizResult.java

```
package com.quizapp.model;

import jakarta.persistence.*;
import lombok.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "quiz_results")
@Getter @Setter @NoArgsConstructor
public class QuizResult {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(optional = false, fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id")
    private User user;

    private int score;
    private int total;
    private LocalDateTime takenAt = LocalDateTime.now();
}
```

backend/src/main/java/com/quizapp/repo/UserRepository.java

```
package com.quizapp.repo;

import com.quizapp.model.User;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByUsername(String username);
    boolean existsByUsername(String username);
}
```

backend/src/main/java/com/quizapp/repo/QuestionRepository.java

```
package com.quizapp.repo;

import com.quizapp.model.Question;
import org.springframework.data.jpa.repository.JpaRepository;

public interface QuestionRepository extends JpaRepository<Question, Long> {}
```

backend/src/main/java/com/quizapp/repo/QuizResultRepository.java

```
package com.quizapp.repo;

import com.quizapp.model.QuizResult;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface QuizResultRepository extends JpaRepository<QuizResult, Long> {
    List<QuizResult> findByUserUsernameOrderByTakenAtDesc(String username);
}
```

Backend - DTOs, security, config, service

backend/src/main/java/com/quizapp/dto/Dtos.java

```
package com.quizapp.dto;

import com.quizapp.model.Question;
import com.quizapp.model.QuizResult;
import jakarta.validation.constraints.*;
import java.time.LocalDateTime;
import java.util.List;
import java.util.Map;

public final class Dtos {
    private Dtos() {}

    public record RegisterRequest(
            @NotBlank @Size(min = 3, max = 30) String username,
            @NotBlank @Size(min = 6, max = 100) String password) {}

    public record LoginRequest(@NotBlank String username, @NotBlank String password) {}

    public record AuthResponse(String token, String username, String role) {}

    /** What quiz takers see: NO correct answer. */
    public record QuestionDto(Long id, String text, List<String> options, String category) {
        public static QuestionDto from(Question q) {
            return new QuestionDto(q.getId(), q.getText(), q.getOptions(), q.getCategory());
        }
    }

    /** What admins see: includes the correct answer. */
    public record QuestionAdminDto(Long id, String text, List<String> options, int correctIndex,
                                   String category, String explanation) {
        public static QuestionAdminDto from(Question q) {
            return new QuestionAdminDto(q.getId(), q.getText(), q.getOptions(), q.getCorrectIndex(),
                    q.getCategory(), q.getExplanation());
        }
    }

    public record QuestionRequest(
            @NotBlank @Size(max = 500) String text,
            @NotNull @Size(min = 2, max = 6) List<@NotBlank String> options,
            @Min(0) int correctIndex,
            @Size(max = 255) String category,
            @Size(max = 500) String explanation) {}

    /** questionId -> selected option index (-1 = skipped) */
    public record SubmitRequest(@NotNull @NotEmpty Map<Long, Integer> answers) {}

    public record ReviewItem(Long questionId, String text, List<String> options, int selectedIndex,
                             int correctIndex, boolean correct, String explanation) {}

    public record SubmitResponse(int score, int total, List<ReviewItem> review) {}

    public record ResultDto(Long id, int score, int total, LocalDateTime takenAt) {
        public static ResultDto from(QuizResult r) {
            return new ResultDto(r.getId(), r.getScore(), r.getTotal(), r.getTakenAt());
        }
    }
}
```

backend/src/main/java/com/quizapp/security/JwtService.java

```
package com.quizapp.security;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

import javax.crypto.SecretKey;
import java.nio.charset.StandardCharsets;
import java.util.Date;

@Component
public class JwtService {
    private final SecretKey key;
    private final long expirationMs;

    public JwtService(@Value("${app.jwt.secret}") String secret,
                      @Value("${app.jwt.expiration-ms}") long expirationMs) {
        this.key = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
        this.expirationMs = expirationMs;
    }

    public String generate(String username, String role) {
        return Jwts.builder()
                .subject(username)
                .claim("role", role)
                .issuedAt(new Date())
                .expiration(new Date(System.currentTimeMillis() + expirationMs))
                .signWith(key)
                .compact();
    }

    public Claims parse(String token) {
        return Jwts.parser().verifyWith(key).build().parseSignedClaims(token).getPayload();
    }
}
```

backend/src/main/java/com/quizapp/security/JwtAuthFilter.java

```
package com.quizapp.security;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;
import java.util.List;

@Component
public class JwtAuthFilter extends OncePerRequestFilter {
    private final JwtService jwt;

    public JwtAuthFilter(JwtService jwt) { this.jwt = jwt; }

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain)
            throws ServletException, IOException {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            try {
                var claims = jwt.parse(header.substring(7));
                var auth = new UsernamePasswordAuthenticationToken(
                        claims.getSubject(), null,
                        List.of(new SimpleGrantedAuthority("ROLE_" + claims.get("role", String.class))))
      ;
                SecurityContextHolder.getContext().setAuthentication(auth);
            } catch (Exception ignored) {
                // invalid/expired token -> request stays unauthenticated -> 401
            }
        }
        chain.doFilter(req, res);
    }
}
```

backend/src/main/java/com/quizapp/config/SecurityConfig.java

```
package com.quizapp.config;

import com.quizapp.security.JwtAuthFilter;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpStatus;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.HttpStatusEntryPoint;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

import java.util.List;

@Configuration
public class SecurityConfig {

    @Bean
    SecurityFilterChain chain(HttpSecurity http, JwtAuthFilter jwtFilter) throws Exception {
        http.csrf(c -> c.disable()) // stateless JWT API, no cookies
            .cors(c -> {})
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(a -> a
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated())
            .exceptionHandling(e -> e.authenticationEntryPoint(new HttpStatusEntryPoint(HttpStatus.UNAUT
      HORIZED)))
            .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }

    @Bean
    PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }

    @Bean
    CorsConfigurationSource corsSource(@Value("${app.cors.allowed-origin}") String origin) {
        CorsConfiguration cfg = new CorsConfiguration();
        cfg.setAllowedOriginPatterns(List.of(origin, "http://localhost:*", "http://127.0.0.1:*"));
        cfg.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
        cfg.setAllowedHeaders(List.of("Authorization", "Content-Type"));
        UrlBasedCorsConfigurationSource src = new UrlBasedCorsConfigurationSource();
        src.registerCorsConfiguration("/api/**", cfg);
        return src;
    }
}
```

backend/src/main/java/com/quizapp/config/DataInitializer.java

```
package com.quizapp.config;

import com.quizapp.model.Question;
import com.quizapp.model.User;
import com.quizapp.repo.QuestionRepository;
import com.quizapp.repo.UserRepository;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.CommandLineRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.crypto.password.PasswordEncoder;

import java.util.ArrayList;
import java.util.List;

@Configuration
public class DataInitializer {

    @Bean
    CommandLineRunner seed(UserRepository users, QuestionRepository questions, PasswordEncoder encoder,
                           @Value("${app.admin.username}") String adminUser,
                           @Value("${app.admin.password}") String adminPass) {
        return args -> {
            if (!users.existsByUsername(adminUser)) {
                User u = new User();
```

```
                u.setUsername(adminUser);
                u.setPassword(encoder.encode(adminPass));
                u.setRole("ADMIN");
                users.save(u);
            }
            if (questions.count() == 0) {
                questions.saveAll(List.of(
                    q("Which keyword is used to inherit a class in Java?", List.of("implements", "extend
      s", "inherits", "super"), 1, "Java", "'extends' creates a subclass."),
                    q("Which hook manages state in a React function component?", List.of("useEffect", "u
      seRef", "useState", "useMemo"), 2, "React", "useState returns a value and a setter."),
                    q("Which annotation marks a Spring Boot REST controller?", List.of("@Controller", "@
      RestController", "@Service", "@Component"), 1, "Spring", "@RestController = @Controller + @Respons
      eBody."),
                    q("Which SQL clause filters rows after grouping?", List.of("WHERE", "GROUP BY", "HAV
      ING", "ORDER BY"), 2, "MySQL", "HAVING filters aggregated groups."),
                    q("What does HTTP status 404 mean?", List.of("Server error", "Unauthorized", "Not Fo
      und", "Created"), 2, "Web", "The resource was not found."),
                    q("Which method adds an element to the end of a JS array?", List.of("push", "pop", "
      shift", "unshift"), 0, "JavaScript", "push appends to the end.")
                ));
            }
        };
    }

    private Question q(String text, List<String> opts, int correct, String cat, String expl) {
        Question q = new Question();
        q.setText(text);
        q.setOptions(new ArrayList<>(opts));
        q.setCorrectIndex(correct);
        q.setCategory(cat);
        q.setExplanation(expl);
        return q;
    }
}
```

backend/src/main/java/com/quizapp/service/QuizService.java

```
package com.quizapp.service;

import com.quizapp.dto.Dtos.*;
import com.quizapp.model.Question;
import com.quizapp.model.QuizResult;
import com.quizapp.repo.QuestionRepository;
import com.quizapp.repo.QuizResultRepository;
import com.quizapp.repo.UserRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.server.ResponseStatusException;

import java.util.*;

@Service
@RequiredArgsConstructor
public class QuizService {
    private final QuestionRepository questions;
    private final QuizResultRepository results;
    private final UserRepository users;

    public List<QuestionDto> randomQuestions(int limit) {
        List<Question> all = new ArrayList<>(questions.findAll());
        Collections.shuffle(all);
        return all.stream().limit(Math.max(1, limit)).map(QuestionDto::from).toList();
    }

    @Transactional
    public SubmitResponse submit(String username, Map<Long, Integer> answers) {
        List<Question> qs = questions.findAllById(answers.keySet());
        if (qs.isEmpty()) throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "No valid answers su
      bmitted");

        int score = 0;
        List<ReviewItem> review = new ArrayList<>();
        for (Question q : qs) {
            int selected = answers.getOrDefault(q.getId(), -1);
            boolean correct = selected == q.getCorrectIndex();
            if (correct) score++;
            review.add(new ReviewItem(q.getId(), q.getText(), q.getOptions(), selected,
                    q.getCorrectIndex(), correct, q.getExplanation()));
```

```
        }

        QuizResult r = new QuizResult();
        r.setUser(users.findByUsername(username).orElseThrow());
        r.setScore(score);
        r.setTotal(qs.size());
        results.save(r);
        return new SubmitResponse(score, qs.size(), review);
    }

    public List<ResultDto> history(String username) {
        return results.findByUserUsernameOrderByTakenAtDesc(username).stream().map(ResultDto::from).toLi
      st();
    }
}
```

Backend - controllers and tests

backend/src/main/java/com/quizapp/web/AuthController.java

```
package com.quizapp.web;

import com.quizapp.dto.Dtos.*;
import com.quizapp.model.User;
import com.quizapp.repo.UserRepository;
import com.quizapp.security.JwtService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.server.ResponseStatusException;

@RestController
@RequestMapping("/api/auth")
@RequiredArgsConstructor
public class AuthController {
    private final UserRepository users;
    private final PasswordEncoder encoder;
    private final JwtService jwt;

    @PostMapping("/register")
    public AuthResponse register(@Valid @RequestBody RegisterRequest req) {
        if (users.existsByUsername(req.username()))
            throw new ResponseStatusException(HttpStatus.CONFLICT, "Username already taken");
        User u = new User();
        u.setUsername(req.username());
        u.setPassword(encoder.encode(req.password()));
        u.setRole("USER"); // admins can never self-register
        users.save(u);
        return new AuthResponse(jwt.generate(u.getUsername(), u.getRole()), u.getUsername(), u.getRole()
      );
    }

    @PostMapping("/login")
    public AuthResponse login(@Valid @RequestBody LoginRequest req) {
        User u = users.findByUsername(req.username())
                .filter(x -> encoder.matches(req.password(), x.getPassword()))
                .orElseThrow(() -> new ResponseStatusException(HttpStatus.UNAUTHORIZED, "Invalid credent
      ials"));
        return new AuthResponse(jwt.generate(u.getUsername(), u.getRole()), u.getUsername(), u.getRole()
      );
    }
}
```

backend/src/main/java/com/quizapp/web/QuizController.java

```
package com.quizapp.web;

import com.quizapp.dto.Dtos.*;
import com.quizapp.service.QuizService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/quiz")
@RequiredArgsConstructor
public class QuizController {
    private final QuizService service;

    @GetMapping("/questions")
    public List<QuestionDto> questions(@RequestParam(defaultValue = "5") int limit) {
        return service.randomQuestions(Math.min(limit, 50));
    }

    @PostMapping("/submit")
    public SubmitResponse submit(@Valid @RequestBody SubmitRequest req, Authentication auth) {
        return service.submit(auth.getName(), req.answers());
    }

    @GetMapping("/results")
    public List<ResultDto> history(Authentication auth) {
        return service.history(auth.getName());
    }
}
```

backend/src/main/java/com/quizapp/web/AdminQuestionController.java

```
package com.quizapp.web;

import com.quizapp.dto.Dtos.*;
import com.quizapp.model.Question;
import com.quizapp.repo.QuestionRepository;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.server.ResponseStatusException;

import java.util.ArrayList;
import java.util.List;

@RestController
@RequestMapping("/api/admin/questions")
@RequiredArgsConstructor
public class AdminQuestionController {
    private final QuestionRepository repo;

    @GetMapping
    public List<QuestionAdminDto> all() {
        return repo.findAll().stream().map(QuestionAdminDto::from).toList();
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public QuestionAdminDto create(@Valid @RequestBody QuestionRequest req) {
        return QuestionAdminDto.from(repo.save(apply(new Question(), req)));
    }

    @PutMapping("/{id}")
    public QuestionAdminDto update(@PathVariable Long id, @Valid @RequestBody QuestionRequest req) {
        Question q = repo.findById(id).orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUN
      D));
        return QuestionAdminDto.from(repo.save(apply(q, req)));
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id) {
        if (!repo.existsById(id)) throw new ResponseStatusException(HttpStatus.NOT_FOUND);
        repo.deleteById(id);
    }
```

```
    private Question apply(Question q, QuestionRequest r) {
        if (r.correctIndex() >= r.options().size())
            throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "correctIndex out of range");
        q.setText(r.text());
        q.setOptions(new ArrayList<>(r.options()));
        q.setCorrectIndex(r.correctIndex());
        q.setCategory(r.category());
        q.setExplanation(r.explanation());
        return q;
    }
}
```

backend/src/main/java/com/quizapp/web/GlobalExceptionHandler.java

```
package com.quizapp.web;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.server.ResponseStatusException;

import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, String>> validation(MethodArgumentNotValidException ex) {
        String msg = ex.getBindingResult().getFieldErrors().stream()
                .map(e -> e.getField() + ": " + e.getDefaultMessage()).findFirst().orElse("Invalid reque
      st");
        return ResponseEntity.badRequest().body(Map.of("message", msg));
    }

    @ExceptionHandler(ResponseStatusException.class)
    public ResponseEntity<Map<String, String>> status(ResponseStatusException ex) {
        return ResponseEntity.status(ex.getStatusCode())
                .body(Map.of("message", ex.getReason() == null ? "Error" : ex.getReason()));
    }
}
```

backend/src/test/java/com/quizapp/QuizServiceTest.java

```
package com.quizapp;

import com.quizapp.dto.Dtos.SubmitResponse;
import com.quizapp.model.Question;
import com.quizapp.model.User;
import com.quizapp.repo.QuestionRepository;
import com.quizapp.repo.QuizResultRepository;
import com.quizapp.repo.UserRepository;
import com.quizapp.service.QuizService;
import org.junit.jupiter.api.Test;
import org.mockito.Mockito;

import java.util.List;
import java.util.Map;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertFalse;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.when;

class QuizServiceTest {

    private Question question(long id, int correct) {
        Question q = new Question();
        q.setId(id);
        q.setText("Q" + id);
        q.setOptions(List.of("a", "b", "c"));
        q.setCorrectIndex(correct);
        return q;
    }

    @Test
    void scoresCorrectAnswersAndTreatsSkippedAsWrong() {
        var qr = Mockito.mock(QuestionRepository.class);
        var rr = Mockito.mock(QuizResultRepository.class);
```

```
        var ur = Mockito.mock(UserRepository.class);
        when(qr.findAllById(any())).thenReturn(List.of(question(1, 0), question(2, 2), question(3, 1)));
        when(ur.findByUsername("bob")).thenReturn(Optional.of(new User()));

        SubmitResponse res = new QuizService(qr, rr, ur).submit("bob", Map.of(1L, 0, 2L, 1, 3L, -1));

        assertEquals(1, res.score());
        assertEquals(3, res.total());
        assertFalse(res.review().get(1).correct());
    }
}
```

Database

database/schema.sql

```
-- Optional: Hibernate (ddl-auto=update) creates these automatically.
-- Provided for documentation / manual setup.
CREATE DATABASE IF NOT EXISTS quizdb;
USE quizdb;

CREATE TABLE users (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  role VARCHAR(20) NOT NULL
);

CREATE TABLE questions (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  text VARCHAR(500) NOT NULL,
  correct_index INT NOT NULL,
  category VARCHAR(255),
  explanation VARCHAR(500)
);

CREATE TABLE question_options (
  question_id BIGINT NOT NULL,
  position INT NOT NULL,
  option_text VARCHAR(300) NOT NULL,
  PRIMARY KEY (question_id, position),
  FOREIGN KEY (question_id) REFERENCES questions(id) ON DELETE CASCADE
);

CREATE TABLE quiz_results (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id BIGINT NOT NULL,
  score INT NOT NULL,
  total INT NOT NULL,
  taken_at DATETIME(6),
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
```

Frontend - configuration

frontend/package.json

```
{
  "name": "quiz-frontend",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": { "dev": "vite", "build": "vite build", "preview": "vite preview" },
  "dependencies": {
    "axios": "^1.7.2",
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "react-router-dom": "^6.23.1"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.3.1",
    "vite": "^5.3.1"
  }
}
```

frontend/vite.config.js

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

// In development the browser talks only to http://localhost:5173.
// Vite forwards every /api request to the Spring Boot backend (port 8081),
// so no cross-origin (CORS) check is needed while developing.
export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    proxy: {
      '/api': {
        target: 'http://localhost:8081',
        changeOrigin: true,
        configure: (proxy) => {
          proxy.on('proxyReq', (proxyReq) => proxyReq.removeHeader('origin'));
        },
      },
    },
  },
});
```

frontend/.env.example

```
VITE_API_URL=/api
```

frontend/index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Quiz App</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

Frontend - application core

frontend/src/main.jsx

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import App from './App.jsx';
import { AuthProvider } from './AuthContext.jsx';
import './styles.css';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <BrowserRouter>
      <AuthProvider>
        <App />
      </AuthProvider>
    </BrowserRouter>
  </React.StrictMode>
);
```

frontend/src/api.js

```
import axios from 'axios';

const api = axios.create({ baseURL: import.meta.env.VITE_API_URL || 'http://localhost:8080/api' });

api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

api.interceptors.response.use(
  (r) => r,
  (err) => {
    // expired/invalid token on a protected call -> force re-login
    if (err.response?.status === 401 && !err.config.url.includes('/auth/')) {
      localStorage.clear();
      window.location.href = '/login';
    }
    return Promise.reject(err);
  }
);

export const errMsg = (e, fallback = 'Something went wrong') => e.response?.data?.message || fallback;
export default api;
```

frontend/src/AuthContext.jsx

```
import { createContext, useContext, useState } from 'react';

const AuthContext = createContext(null);
export const useAuth = () => useContext(AuthContext);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(() => {
    const token = localStorage.getItem('token');
    return token ? { token, username: localStorage.getItem('username'), role: localStorage.getItem('role
      ') } : null;
  });

  const login = ({ token, username, role }) => {
    localStorage.setItem('token', token);
    localStorage.setItem('username', username);
    localStorage.setItem('role', role);
    setUser({ token, username, role });
  };
  const logout = () => {
    localStorage.clear();
    setUser(null);
  };

  return <AuthContext.Provider value={{ user, login, logout }}>{children}</AuthContext.Provider>;
}
```

frontend/src/App.jsx

```
import { Navigate, Route, Routes } from 'react-router-dom';
import { useAuth } from './AuthContext.jsx';
import Navbar from './components/Navbar.jsx';
import Login from './pages/Login.jsx';
import Home from './pages/Home.jsx';
import Quiz from './pages/Quiz.jsx';
import Result from './pages/Result.jsx';
import History from './pages/History.jsx';
import Admin from './pages/Admin.jsx';

function Protected({ children, admin = false }) {
  const { user } = useAuth();
  if (!user) return <Navigate to="/login" replace />;
  if (admin && user.role !== 'ADMIN') return <Navigate to="/" replace />;
  return children;
}

export default function App() {
  return (
    <>
      <Navbar />
      <main className="container">
        <Routes>
          <Route path="/login" element={<Login />} />
```

```
          <Route path="/" element={<Protected><Home /></Protected>} />
          <Route path="/quiz" element={<Protected><Quiz /></Protected>} />
          <Route path="/result" element={<Protected><Result /></Protected>} />
          <Route path="/history" element={<Protected><History /></Protected>} />
          <Route path="/admin" element={<Protected admin><Admin /></Protected>} />
          <Route path="*" element={<Navigate to="/" replace />} />
        </Routes>
      </main>
    </>
  );
}
```

frontend/src/components/Navbar.jsx

```
import { Link, useNavigate } from 'react-router-dom';
import { useAuth } from '../AuthContext.jsx';

export default function Navbar() {
  const { user, logout } = useAuth();
  const nav = useNavigate();
  return (
    <nav className="navbar">
      <Link to="/" className="brand">\u{1F9E0} QuizApp</Link>
      {user && (
        <div className="links">
          <Link to="/history">My Scores</Link>
          {user.role === 'ADMIN' && <Link to="/admin">Admin</Link>}
          <span className="muted">{user.username}</span>
          <button className="btn small secondary" onClick={() => { logout(); nav('/login'); }}>Logout</b
      utton>
        </div>
      )}
    </nav>
  );
}
```

Frontend - pages

frontend/src/pages/Login.jsx

```
import { useState } from 'react';
import { Navigate, useNavigate } from 'react-router-dom';
import api, { errMsg } from '../api.js';
import { useAuth } from '../AuthContext.jsx';

export default function Login() {
  const { user, login } = useAuth();
  const nav = useNavigate();
  const [mode, setMode] = useState('login');
  const [form, setForm] = useState({ username: '', password: '' });
  const [error, setError] = useState('');

  if (user) return <Navigate to="/" replace />;

  const submit = async (e) => {
    e.preventDefault();
    setError('');
    try {
      const { data } = await api.post(`/auth/${mode}`, form);
      login(data);
      nav('/');
    } catch (err) {
      setError(errMsg(err, 'Request failed'));
    }
  };

  return (
    <div className="card narrow">
      <h2>{mode === 'login' ? 'Login' : 'Create account'}</h2>
      <form onSubmit={submit}>
        <label>Username</label>
        <input value={form.username} onChange={(e) => setForm({ ...form, username: e.target.value })} re
      quired />
        <label>Password</label>
        <input type="password" value={form.password} onChange={(e) => setForm({ ...form, password: e.tar
      get.value })} required />
        {error && <p className="error">{error}</p>}
        <button className="btn" type="submit">{mode === 'login' ? 'Login' : 'Register'}</button>
      </form>
```

```
      <p className="muted">
        {mode === 'login' ? 'New here? ' : 'Already registered? '}
        <a href="#" onClick={(e) => { e.preventDefault(); setMode(mode === 'login' ? 'register' : 'login
      '); setError(''); }}>
          {mode === 'login' ? 'Create an account' : 'Login'}
        </a>
      </p>
    </div>
  );
}
```

frontend/src/pages/Home.jsx

```
import { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { useAuth } from '../AuthContext.jsx';

export default function Home() {
  const { user } = useAuth();
  const nav = useNavigate();
  const [count, setCount] = useState(5);
  return (
    <div className="card narrow center">
      <h2>Welcome, {user.username}!</h2>
      <p className="muted">Pick how many questions you want and test yourself.</p>
      <select value={count} onChange={(e) => setCount(Number(e.target.value))}>
        {[3, 5, 10, 15].map((n) => <option key={n} value={n}>{n} questions</option>)}
      </select>
      <button className="btn" onClick={() => nav('/quiz', { state: { count } })}>Start Quiz</button>
    </div>
  );
}
```

frontend/src/pages/Quiz.jsx

```
import { useEffect, useState } from 'react';
import { useLocation, useNavigate } from 'react-router-dom';
import api, { errMsg } from '../api.js';

export default function Quiz() {
  const { state } = useLocation();
  const count = state?.count || 5;
  const nav = useNavigate();
  const [questions, setQuestions] = useState([]);
  const [index, setIndex] = useState(0);
  const [answers, setAnswers] = useState({});
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    api.get('/quiz/questions', { params: { limit: count } })
      .then((r) => setQuestions(r.data))
      .catch((e) => setError(errMsg(e, 'Could not load questions')))
      .finally(() => setLoading(false));
  }, [count]);

  if (loading) return <p>Loading…</p>;
  if (error) return <p className="error">{error}</p>;
  if (!questions.length) return <p>No questions available yet. Ask an admin to add some.</p>;

  const q = questions[index];
  const isLast = index === questions.length - 1;
  const answered = Object.keys(answers).length;

  const submit = async () => {
    const payload = Object.fromEntries(questions.map((x) => [x.id, answers[x.id] ?? -1]));
    try {
      const { data } = await api.post('/quiz/submit', { answers: payload });
      nav('/result', { state: data });
    } catch (e) {
      setError(errMsg(e, 'Submit failed'));
    }
  };

  return (
    <div className="card">
      <div className="progress"><div style={{ width: `${((index + 1) / questions.length) * 100}%` }} /><
      /div>
      <p className="muted">Question {index + 1} of {questions.length}{q.category ? ` · ${q.category}` :
      ''}</p>
```

```
      <h2>{q.text}</h2>
      <div className="options">
        {q.options.map((opt, i) => (
          <button key={i} className={`option ${answers[q.id] === i ? 'selected' : ''}`}
                  onClick={() => setAnswers({ ...answers, [q.id]: i })}>
            {opt}
          </button>
        ))}
      </div>
      <div className="row">
        <button className="btn secondary" disabled={index === 0} onClick={() => setIndex(index - 1)}>Pre
      vious</button>
        {isLast
          ? <button className="btn" onClick={submit}>Submit ({answered}/{questions.length} answered)</bu
      tton>
          : <button className="btn" onClick={() => setIndex(index + 1)}>Next</button>}
      </div>
    </div>
  );
}
```

frontend/src/pages/Result.jsx

```
import { Link, Navigate, useLocation } from 'react-router-dom';

export default function Result() {
  const { state } = useLocation();
  if (!state) return <Navigate to="/history" replace />;
  const pct = Math.round((state.score / state.total) * 100);

  return (
    <>
      <div className="card center">
        <h2>Your Score</h2>
        <div className="score">{state.score} / {state.total}</div>
        <p className="muted">{pct}% — {pct >= 80 ? 'Excellent! \u{1F389}' : pct >= 50 ? 'Good effort \u{
      1F44D}' : 'Keep practicing \u{1F4AA}'}</p>
        <Link className="btn" to="/">Try again</Link>
      </div>

      <h3>Review answers</h3>
      {state.review.map((r, n) => (
        <div key={r.questionId} className={`card review ${r.correct ? 'ok' : 'bad'}`}>
          <strong>{n + 1}. {r.text}</strong>
          <ul>
            {r.options.map((o, i) => (
              <li key={i} className={i === r.correctIndex ? 'correct' : i === r.selectedIndex ? 'wrong'
      : ''}>
                {o}
                {i === r.correctIndex && ' \u{2713}'}
                {i === r.selectedIndex && i !== r.correctIndex && ' \u{2717} (your answer)'}
              </li>
            ))}
          </ul>
          {r.selectedIndex === -1 && <p className="muted">You skipped this question.</p>}
          {r.explanation && <p className="muted">\u{1F4A1} {r.explanation}</p>}
        </div>
      ))}
    </>
  );
}
```

frontend/src/pages/History.jsx

```
import { useEffect, useState } from 'react';
import api from '../api.js';

export default function History() {
  const [rows, setRows] = useState(null);
  useEffect(() => { api.get('/quiz/results').then((r) => setRows(r.data)); }, []);
  if (!rows) return <p>Loading…</p>;
  return (
    <div className="card">
      <h2>My Scores</h2>
      {rows.length === 0 ? <p className="muted">No attempts yet.</p> : (
        <table>
          <thead><tr><th>Date</th><th>Score</th><th>%</th></tr></thead>
          <tbody>
            {rows.map((r) => (
              <tr key={r.id}>
```

```
              <td>{new Date(r.takenAt).toLocaleString()}</td>
              <td>{r.score} / {r.total}</td>
              <td>{Math.round((r.score / r.total) * 100)}%</td>
            </tr>
          ))}
        </tbody>
      </table>
    )}
  </div>
 );
}
```

frontend/src/pages/Admin.jsx

```
import { useEffect, useState } from 'react';
import api, { errMsg } from '../api.js';

const empty = { text: '', options: ['', '', '', ''], correctIndex: 0, category: '', explanation: '' };

export default function Admin() {
  const [questions, setQuestions] = useState([]);
  const [form, setForm] = useState(empty);
  const [editingId, setEditingId] = useState(null);
  const [error, setError] = useState('');

  const load = () => api.get('/admin/questions').then((r) => setQuestions(r.data));
  useEffect(() => { load(); }, []);

  const setOption = (i, v) => setForm({ ...form, options: form.options.map((o, k) => (k === i ? v : o))
    });

  const save = async (e) => {
    e.preventDefault();
    setError('');
    const options = form.options.map((o) => o.trim()).filter(Boolean);
    if (form.correctIndex >= options.length || !form.options[form.correctIndex].trim()) {
      return setError('Correct answer must point to a filled option.');
    }
    const body = { ...form, options };
    try {
      editingId ? await api.put(`/admin/questions/${editingId}`, body) : await api.post('/admin/question
      s', body);
      setForm(empty);
      setEditingId(null);
      load();
    } catch (err) {
      setError(errMsg(err));
    }
  };

  const edit = (q) => {
    setEditingId(q.id);
    setForm({ ...q, options: [...q.options, '', '', '', ''].slice(0, Math.max(4, q.options.length)), cat
      egory: q.category || '', explanation: q.explanation || '' });
    window.scrollTo({ top: 0, behavior: 'smooth' });
  };

  const remove = async (id) => {
    if (!confirm('Delete this question?')) return;
    await api.delete(`/admin/questions/${id}`);
    load();
  };

  return (
    <>
      <div className="card">
        <h2>{editingId ? 'Edit question' : 'Add question'}</h2>
        <form onSubmit={save}>
          <label>Question</label>
          <textarea value={form.text} onChange={(e) => setForm({ ...form, text: e.target.value })} requi
      red />
          <label>Options (select the correct one)</label>
          {form.options.map((o, i) => (
            <div className="row" key={i}>
              <input type="radio" name="correct" checked={form.correctIndex === i} onChange={() => setFo
      rm({ ...form, correctIndex: i })} />
              <input value={o} placeholder={`Option ${i + 1}`} onChange={(e) => setOption(i, e.target.va
      lue)} required={i < 2} />
            </div>
          ))}
          <label>Category</label>
```

```
          <input value={form.category} onChange={(e) => setForm({ ...form, category: e.target.value })}
      />
          <label>Explanation (shown in review)</label>
          <input value={form.explanation} onChange={(e) => setForm({ ...form, explanation: e.target.valu
      e })} />
          {error && <p className="error">{error}</p>}
          <div className="row">
            <button className="btn" type="submit">{editingId ? 'Update' : 'Add'}</button>
            {editingId && <button type="button" className="btn secondary" onClick={() => { setEditingId(
      null); setForm(empty); }}>Cancel</button>}
          </div>
        </form>
      </div>

      <div className="card">
        <h2>All questions ({questions.length})</h2>
        <table>
          <thead><tr><th>Question</th><th>Category</th><th>Correct</th><th></th></tr></thead>
          <tbody>
            {questions.map((q) => (
              <tr key={q.id}>
                <td>{q.text}</td>
                <td>{q.category}</td>
                <td>{q.options[q.correctIndex]}</td>
                <td className="actions">
                  <button className="btn small secondary" onClick={() => edit(q)}>Edit</button>
                  <button className="btn small danger" onClick={() => remove(q.id)}>Delete</button>
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    </>
  );
}
```

Frontend - styles and repository files

frontend/src/styles.css

```
:root { --primary:#4f46e5; --bg:#f3f4f6; --card:#fff; --text:#111827; --muted:#6b7280; --ok:#16a34a; --b
      ad:#dc2626; }
* { box-sizing: border-box; }
body { margin:0; font-family: system-ui, sans-serif; background:var(--bg); color:var(--text); }
.navbar { display:flex; justify-content:space-between; align-items:center; padding:12px 24px; background
      :var(--card); box-shadow:0 1px 4px rgba(0,0,0,.08); }
.navbar .links { display:flex; gap:16px; align-items:center; }
.brand { font-weight:700; font-size:1.2rem; text-decoration:none; color:var(--primary); }
a { color:var(--primary); }
.container { max-width:800px; margin:24px auto; padding:0 16px; }
.card { background:var(--card); padding:24px; border-radius:12px; box-shadow:0 1px 4px rgba(0,0,0,.08);
      margin-bottom:16px; }
.narrow { max-width:420px; margin:40px auto; }
.center { text-align:center; }
.muted { color:var(--muted); }
.error { color:var(--bad); }
label { display:block; margin:12px 0 4px; font-weight:600; font-size:.9rem; }
input, textarea, select { width:100%; padding:10px; border:1px solid #d1d5db; border-radius:8px; font:in
      herit; }
input[type=radio] { width:auto; }
.row { display:flex; gap:8px; align-items:center; margin-top:8px; justify-content:space-between; }
.btn { display:inline-block; background:var(--primary); color:#fff; border:0; padding:10px 18px; border-
      radius:8px; cursor:pointer; font:inherit; text-decoration:none; margin-top:12px; }
.btn:disabled { opacity:.5; cursor:not-allowed; }
.btn.secondary { background:#e5e7eb; color:var(--text); }
.btn.danger { background:var(--bad); }
.btn.small { padding:4px 10px; margin:0; font-size:.85rem; }
.options { display:grid; gap:10px; margin:16px 0; }
.option { text-align:left; padding:12px; border:2px solid #e5e7eb; background:#fff; border-radius:8px; c
      ursor:pointer; font:inherit; }
.option:hover { border-color:var(--primary); }
.option.selected { border-color:var(--primary); background:#eef2ff; }
.progress { height:6px; background:#e5e7eb; border-radius:3px; overflow:hidden; margin-bottom:12px; }
.progress div { height:100%; background:var(--primary); transition:width .3s; }
.score { font-size:3rem; font-weight:800; color:var(--primary); }
.review.ok { border-left:5px solid var(--ok); } .review.bad { border-left:5px solid var(--bad); }
.review ul { list-style:none; padding:0; } .review li { padding:6px 10px; border-radius:6px; }
li.correct { background:#dcfce7; } li.wrong { background:#fee2e2; }
table { width:100%; border-collapse:collapse; } th, td { text-align:left; padding:8px; border-bottom:1px
```

```
       solid #e5e7eb; }
.actions { display:flex; gap:6px; }
```

.gitignore

```
node_modules/
dist/
target/
.env
.idea/
.vscode/
*.log
```

Appendix B. README.md (full text)

This is the README file included in the GitHub repository.

README.md

````
# Interactive Quiz Application

**Author:** Ch. Lipika | Full Stack Project 1 - React, Spring Boot, MySQL

A full-stack quiz platform. Users register, take randomized quizzes, see their score and review every
answer (with explanations). Admins manage the question bank from a protected admin panel.

## Features
- **Quiz takers:** register/login, choose quiz length, move between questions, submit, instant score,
  per-question review with explanations, personal score history
- **Admin panel:** add, edit and delete questions (2-6 options, correct answer, category, explanation)
- **Security:** BCrypt password hashing, stateless JWT authentication, role-based access
  (`/api/admin/**` is ADMIN-only), bean validation, correct answers never sent before submission,
  scoring done on the server, admins cannot be created through public registration

## Tech stack
| Layer | Technology |
|---|---|
| Frontend | React 18, Vite 5, React Router 6, Axios |
| Backend | Java 17, Spring Boot 3.2.5, Spring Security, Spring Data JPA, JJWT 0.12.5 |
| Database | MySQL 8 |

## Project structure
```
quiz-app/
  backend/     Spring Boot REST API (Maven)
  frontend/    React single-page app (Vite)
  database/    schema.sql (reference copy of the schema)
```

## Database design
- `users` (id, username UNIQUE, password hash, role)
- `questions` (id, text, correct_index, category, explanation)
- `question_options` (question_id -> questions, position, option_text)
- `quiz_results` (id, user_id -> users, score, total, taken_at)

## Setup

### Prerequisites
Java 17+, Maven 3.9+, Node.js 18+, MySQL 8 (running), Git.
On Windows make sure `JAVA_HOME` points to JDK 17 and Maven's `bin` folder is on PATH.

### 1. Backend (port 8081)
```powershell
cd backend
$env:DB_USER='root'
$env:DB_PASSWORD='your_mysql_password'
mvn spring-boot:run
```
Wait for `Started QuizApplication`. The database `quizdb` and its tables are created automatically.
On first start an admin account and six sample questions are seeded:

| Username | Password |
|---|---|
| `admin` | `admin123` |

To use another port: `$env:SERVER_PORT='8082'` (and update the `target` in `frontend/vite.config.js`).

### 2. Frontend (port 5173)
```powershell
cd frontend
copy .env.example .env
npm install
npm run dev
```
Open http://localhost:5173. In development Vite proxies every `/api` request to
`http://localhost:8081`, so the browser never makes a cross-origin call.
Restart `npm run dev` after editing `.env` or `vite.config.js`.

### 3. Tests
```powershell
cd backend
mvn test
```
````

````
## API reference
Protected routes need the header `Authorization: Bearer <token>`.

| Method | Endpoint | Access | Description |
|---|---|---|---|
| POST | `/api/auth/register` | public | `{username, password}` returns `{token, username, role}` |
| POST | `/api/auth/login` | public | `{username, password}` returns `{token, username, role}` |
| GET | `/api/quiz/questions?limit=5` | user | Random questions (without answers) |
| POST | `/api/quiz/submit` | user | `{answers:{"<questionId>": selectedIndex}}` (-1 = skipped), returns score and review |
| GET | `/api/quiz/results` | user | Current user's attempt history |
| GET | `/api/admin/questions` | admin | All questions including answers |
| POST | `/api/admin/questions` | admin | Create a question |
| PUT | `/api/admin/questions/{id}` | admin | Update a question |
| DELETE | `/api/admin/questions/{id}` | admin | Delete a question |

Question body: `{ "text": "...", "options": ["A","B","C","D"], "correctIndex": 1, "category": "Java", "explanation": "..." }`

## Deployment
**Backend**
1. `cd backend && mvn clean package`
2. Run `java -jar target/quiz-backend-1.0.0.jar` with these environment variables:
   `DB_USER`, `DB_PASSWORD`, `JWT_SECRET` (32+ random characters), `ADMIN_PASSWORD`,
   `CORS_ORIGIN` (the URL of your deployed frontend), `SERVER_PORT`.
3. Point `spring.datasource.url` at your hosted MySQL (for example via `SPRING_DATASOURCE_URL`).

**Frontend**
1. Set `VITE_API_URL` to the public API address, e.g. `https://api.example.com/api`
   (or keep `/api` and let Nginx forward `/api` to the backend).
2. `npm run build`, then host the `dist/` folder (Netlify, Vercel, Nginx, ...).
   Configure a fallback to `index.html` so React Router URLs work on refresh.

The Vite proxy is for local development only; in production the browser calls the API address
directly, so `CORS_ORIGIN` on the backend must match the frontend's URL.

## Troubleshooting
| Problem | Fix |
|---|---|
| `release version 17 not supported` | Maven uses the wrong JDK. Check `mvn -v` shows Java 17 and fix `JAVA_HOME`. |
| `Port 8081 was already in use` | An old backend is still running: `netstat -ano \| findstr :8081`, then `taskkill /PID <pid> /F`. |
| `Access denied for user 'root'` | `DB_PASSWORD` does not match your MySQL password. |
| Login shows "Request failed" | Backend not running, or `.env` / `vite.config.js` changed without restarting `npm run dev`. |
| `npm` says scripts are disabled | `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` |

## Version control
```bash
git init
git add .
git commit -m "Interactive quiz application"
git branch -M main
git remote add origin https://github.com/<username>/<repo>.git
git push -u origin main
```

---
Developed by **Ch. Lipika**.
````